1 Time to Event and Censoring

We want to analyze the time until an event occurs. Modelling survival times can involve three statistical approaches: parametric, semiparametric, and non-parametric. Removing censored data will result in estimation uncertainty to be larger than it could be if we were to include the censored data. Under a frequentist approach, we would lose data points which will decrease the precision of our estimates. Removing censored data could also result in biased estimates if data have only been collected for a short time. These biased estimates would be obtained from those observations where the event of interest actually happened. In fact, if you didn't consider censored observations, your survival estimates would be biased and would likely underestimate how long people actually live.

Random Right-censoring: if a patient were hit by a bus and died in the accident, the death would be unrelated to cancer. Hence, all we know is that the patient did not die due to cancer until that moment.

Left-censoring: Left-censoring happens when the event of interest occurred before you started observing someone, so you know the event time is earlier than a certain point, but you do not know exactly when. suppose you are studying when kids learn how to ski and you begin collecting data today. One of the kids in your study already knows how to ski at the first observation.

1.1 Survival Function

In short, the survival function $S_Y(t)$ is the probability that an event has not yet occurred by time t.Key RelationshipWhile the standard CDF ($F_Y(t)$) measures the probability that an event has happened, the survival function measures the probability of "survival" beyond that time:$S_Y(t) = P(Y > t) = 1 - F_Y(t)$ Any survival function (measured as a probability on the x-axis with time on the y-axis) will always be monotonically decreasing as time goes by. This behaviour is expected in any subject since their survival chances would decrease over time.

1.2 Hazard Function

The hazard function (or hazard rate) describes the "instantaneous" risk of an event happening at a specific moment, provided the subject has survived up to that point. the hazard function zooms in on the current risk of "failing" right now. in survival function.

1.3 Weibull Distribution

is one of the most popular parametric modelling choices in Survival Analysis to model. We can obtain the survival function for any positive random variable. (because time is positive always)

$f(t;\alpha,\beta) = \frac{\alpha}{\beta}\left(\frac{t}{\beta}\right)^{\alpha-1} e^{-(t/\beta)^{\alpha}}, \quad t \geq 0$ where α is called the shape parameter and β is the scale parameter. Note that if $\alpha = 1$ we get the exponential distribution with parameter β

Hazard function $h(t) = \frac{\alpha}{\beta}\left(\frac{t}{\beta}\right)^{\alpha-1}$ To get the survival probability of each time t:-

```
pweibull(q = 2, shape = 1,scale = 1,lower.tail = FALSE)
```

2 Estimating the Survival Function

There are two options for estimating quantities by incorporating the partial information contained in censored observations in the form of an estimated survival function of the population of interest: **Parametric**: If a distributional assumption is made, we can use censored likelihood-based methods to estimate the parameters of the chosen heavy-tailed PDF for the observed survival times (e.g., Weibull). Therefore, any quantity (such as the theoretical mean or specific quantiles) can be extracted from that chosen distribution

2.1 Non-parametric=Surv func with Kaplan-Meier

Training dataset of survival times, but we have no distributional assumption of survival function. This option will implicate the following: **RESTRICTED mean**: it can be estimated as the area under an estimate of the survival function. **Quantiles**: they can be estimated by inverting an estimate of the survival function. The tidy() output provides eight metrics as columns from our training data.

```
fit_km <- survfit(formula = Surv(col1, col2) ~ 1,data = data)
tidy(fit_km) #survival function
quantile(fit_km,probs = 0.5,conf.int = FALSE) #median surv time
```

2.2 Parametric-Surv func with Weibull

We need the corresponding parameters of the distribution. estimate the parameters of the Weibull using survreg() **Weibull**: The hazard is monotonic. It either always increases, always decreases, or stays constant. If you believe your patients' risk of death only gets higher as time goes on (due to aging or disease progression), Weibull is a strong candidate. **Lognormal**: The hazard is non-monotonic (arc-shaped). It typically rises to a peak and then decreases toward zero for very long-term survivors. This is common in cases where if a patient survives the initial dangerous period (like right after a high-risk surgery), their long-term risk actually drops.

```
web_model <- survreg(formula = Surv(col1, col2) ~ 1,dist =
     "weibull", data = data)
intercept <- as.numeric(coef(fit_weibull))
log_scale <- as.numeric(log(fit_weibull$scale))
weibull_shape <- 1 / fit_weibull$scale
weibull_scale <- exp(intercept)
median_weibull <- qweibull(p = 0.5, shape = weibull_shape, scale
     = weibull_scale)
mean_weibull <- weibull_scale * gamma(1+(1/weibull_shape))
```

2.3 Semi-Parametric-Cox Proportional Hazards Model

Do not assume any specific distribution for our response of interest. Approach in the form of a widely popular semiparametric regression model. is done through another special maximum likelihood technique using a partial likelihood. A partial likelihood is a specific class of quasi-likelihood, which does not require assuming any specific PDF for the continuous survival times.

```
fit_ph <- coxph(formula = Surv(col1, col2) ~ j_reg, data = data)
tidy(fit_ph, exponentiate = TRUE, conf.int = TRUE)
profile_data <- data.frame(ph.ecog = factor(2, levels = c(0, 1,
     2)), meal.cal = 800)
fit_specific <- survfit(fit_cox, newdata = profile_data)
autoplot(fit_specific, conf.int = TRUE, surv.colour = "red") +
 labs( title = "Label", x = "Time (Days)", y = "Survival
     Probability")
```

a small enough p-value (less than the significance level) indicates that the data provides evidence in favour of association between the hazard function and the jth regressor. **Interpretation:** Now, the interpretation for the regression coefficient in a categorical regressor is the increase (at any time) by $exp(\beta_j)$ times in hazard from the baseline category to another given category.

3 Piecewise Constant Regression

Goal is an accurate prediction in many complex frameworks (e.g., non-linear). fitting different **local OLS regressions**. The **step function** breaks the ranges of the regressor into intervals. In each interval, we adjust a constant. We obtain the average response in that interval. The regression model becomes $Y_i = \beta_0 + \beta_1 C_1(X_i) + \beta_2 C_2(X_i) + \cdots + \beta_{a-1} C_{a-1}(X_i) + \beta_a C_a(X_i) + \varepsilon_i$

```
# Define the number of step functions (q + 1).
breaks_q <- 5
fat_content <- fat_content |> mutate(steps = cut(fat_content$week,
     breaks = breaks_q, right = FALSE))
levels(fat_content$steps)
model_steps <- lm(fat ~ steps, data = fat_content)
tidy(model_steps) |> mutate_if(is.numeric, round, 3)
glance(model_steps)
```

4 Non continuous Piecewise LR

So, for each interval beginning c1, we have an additional intercept and an additional slope.

```
model_piecewise_linear <- lm(fat ~ week * steps, data = fat_content)
```

5 Continuous Piecewise Linear Regression

To fix the above matter, we can impose restrictions to make sure that the lines are connected to each other. Therefore, we need to introduce the concept of the hinge function.

```
levels(fat_content$steps)
knots <- c(7.8, 14.6, 21.4, 28.2)
model_piecewise_cont_linear <- lm(
  fat ~ week +
    I((week - knots[1]) * (week >= knots[1])) +
    I((week - knots[2]) * (week >= knots[2])) +
    I((week - knots[3]) * (week >= knots[3])) +
    I((week - knots[4]) * (week >= knots[4])),
  data = fat_content
)
glance(model_piecewise_cont_linear)
```

6 k-nn Regression

The idea behind K-NN regression is finding the K observations in our training dataset of size n (with p features x_j per observation) closest to the new point we want to predict.

```
# Use odd numbers here so there is no missmatch on how the ties are
     handled.
k <- 7
my_knn <- knnreg(fat ~ week, data = fat_content, k = k)
grid <- fat_content |> data_grid(week) |> add_predictions(my_knn)
```

7 (LOWESS) Regression

Predict response y for a specific x (e.g., fat content at week 10).Define Neighborhood: Identify the closest data points to the target x. The number of points (span) is a user-defined parameter.Assign Weights: Apply weights to these neighbors based on distance. Points closer to the target x receive higher weights (w_i), while distant points receive lower weights.Local Regression: Perform a Weighted Least Squares (WLS) fit using a polynomial (often 1st or 2nd degree). For a second-degree polynomial, minimize the weighted sum of squared errors: $\sum_i w_i(y_i - \beta_0 - \beta_1 x_i - \beta_2 x_i^2)^2$

```
p <- list()
i <- 1
for (degree in seq(0, 2, 1)) {
  for (span in seq(0.25, 1, 0.25)) {
    model_loess <- loess(fat ~ week,
      span = span, degree = degree,
      data = fat_content
    )
    i <- i + 1
  }
}
```

8 Parametric Quantile Regression

How does the alcohol price affect the weekly consumption of light drinkers, moderate drinkers, and heavy drinkers at given cut-off values of alcohol consumption? How does the alcohol price (X) affect the weekly consumption (Y) of light drinkers ($\tau = 0.25$), moderate drinkers ($\tau = 0.5$), and heavy drinkers ($\tau = 0.9$) in given cut-off values in weekly alcohol consumption? This prediction will not be a confidence interval since we are talking about probabilities (i.e., chances). Therefore, we can use "probability" instead of "confidence." Moreover, we would obtain prediction bands.